Questions de soutenance — Marketplace ASMAD

Préparation à l'oral : formulations type jury, réponses structurées. Stack : **Vue 3** (SPA, Vue Router, vue-i18n, Axios, Bootstrap), **NestJS 11**, **TypeORM**, **MySQL**. **Images :** pas d'utilisation de Cloudinary ni d'autre CDN ; upload via l'API, encodage et persistance conformément à votre implémentation actuelle (voir section 8).

Sommaire des thèmes

- 1. Problème métier, objectifs et acteurs
- 2. Architecture logicielle et choix d'ingénierie
- 3. Couche présentation (Vue 3)
- 4. Couche application (NestJS, API)
- 5. Modèle de données et règles métier
- 6. Sécurité : authentification, autorisation, menaces
- 7. Parcours d'achat, panier, commande
- 8. Fichiers, images (sans Cloudinary), e-mail, social vendeur
- 9. Assistance conversationnelle et IA (OpenAI)
- 10. Pilotage, tableaux de bord, indicateurs
- 11. Qualité logicielle, performance, industrialisation
- 12. Méthodologie agile (Scrum) et collaboration
- 13. Déploiement, exploitation, continuité de service
- 14. Bilan critique et feuille de route

1. Problème métier, objectifs et acteurs

Q1. En quoi consiste concrètement votre projet, et quel besoin métier adresse-t-il ?

R. Il s'agit d'une **marketplace** : une plateforme où plusieurs **vendeurs** publient des produits, où des **clients** consultent un catalogue, constituent un panier et passent des commandes, et où des **administrateurs** assurent la cohérence des données, la gestion des comptes et le suivi global. La marque présentée côté interface est **ASMAD**. Le besoin est de **mutualiser l'offre** tout en **séparant les responsabilités** selon le rôle.

Q2. Pourquoi ne pas s'appuyer uniquement sur une boutique « mono-vendeur » (solution classique) ?

R. Une marketplace permet de **faire coexister plusieurs catalogues** et identités vendeurs sous une même expérience utilisateur, ce qui complexifie le modèle (rôles, droits, agrégation des produits) mais reflète mieux des cas réels (places de marché, artisans, PME multiples).

Q3. Pouvez-vous détailler les acteurs, leurs objectifs et leurs contraintes respectives ?

R. **Client :** découvrir, comparer, acheter, suivre ses commandes, éventuellement noter et **suivre** des vendeurs. **Vendeur :** gérer ses produits et son image, lire les retours. **Administrateur :** superviser utilisateurs, catégories globales, commandes, contenus sensibles. Chaque acteur accède à un **sous-ensemble de fonctionnalités** contrôlé côté API (rôles).

Q4. Qu'avez-vous volontairement laissé hors périmètre ou traité de façon simplifiée ?

R. À citer selon votre démo : **paiement en ligne** avec un prestataire agréé (PSP) en conditions réelles, **facturation normative**, **logistique** (transport, étiquettes), **haute disponibilité multi-site**, et éventuellement une **suite de tests automatisés** couvrant tous les parcours. L'important est d'assumer clairement le périmètre et d'indiquer la **prochaine étape** pour industrialiser.

2. Architecture logicielle et choix d'ingénierie

Q5. Quels sont les avantages d'une séparation stricte entre interface (SPA) et API REST ?

R. On obtient un **contrat d'interface** (HTTP/JSON) stable entre les couches : le front peut évoluer (UX, framework) sans réécrire la logique métier, et d'autres clients (mobile, partenaires) pourraient consommer la même API. Cela facilite aussi le **déploiement indépendant** et les tests ciblés.

Q6. Pourquoi avoir retenu NestJS plutôt qu'un framework minimaliste type Express seul ?

R. NestJS impose une **architecture par modules**, l'**injection de dépendances** et des patterns éprouvés (guards, pipes, interceptors), ce qui améliore la **lisibilité** et la **scalabilité humaine** du projet (onboarding, revues). L'intégration officielle avec la validation, JWT et TypeORM réduit le code « plomberie ».

Q7. Qu'apporte Vue 3 dans votre contexte par rapport à d'autres frameworks front ?

R. Vue 3 combine courbe d'apprentissage raisonnable, **Composition API** pour la réutilisabilité, et un écosystème mature (**Vue Router**, **vue-i18n**). Pour une SPA métier avec plusieurs espaces (public, vendeur, admin), la composition par **layouts** et routes imbriquées est adaptée.

Q8. Décrivez la chaîne bout-en-bout d'une requête utilisateur authentifiée.

R. Le navigateur exécute la SPA Vue ; le routeur affiche la vue protégée si le jeton est présent ; **Axios** envoie la requête avec `Authorization: Bearer <JWT>` ; NestJS applique **Passport-JWT** et les **guards** de rôle ; le contrôleur délègue au **service** ; TypeORM interroge **MySQL** ; la réponse JSON remonte jusqu'au composant qui met à jour l'interface.

Q9. Où se situent la logique métier et pourquoi ne pas la laisser uniquement dans le front ?

R. La logique critique (prix, statuts, droits, intégrité des commandes) doit vivre **côté serveur** : le client peut être modifié par l'utilisateur. Le front valide l'UX ; le back **valide et applique** les règles et la persistance.

3. Couche présentation (Vue 3)

Q10. Comment structurez-vous la navigation et la cohérence visuelle entre espaces (client, vendeur, admin) ?

R. **Vue Router** définit des arborescences par domaine fonctionnel ; des **layouts** distincts (par ex. DefaultLayout, SellerLayout, AdminLayout, AuthLayout) encapsulent en-têtes, menus et zones de contenu, ce qui évite la duplication et clarifie les parcours.

Q11. Comment industrialisez-vous le multilingue sans dupliquer les écrans ?

R. Avec **vue-i18n** : les chaînes sont externalisées (fichiers de messages). L'ajout d'une langue consiste à enrichir les ressources, pas à recopier les composants.

Q12. Comment gérez-vous les appels API, les erreurs réseau et l'état de chargement ?

R. Via **Axios** (promesses), avec gestion des codes HTTP (4xx/5xx), messages utilisateur (toasts ou équivalent), et indicateurs de chargement dans les vues sensibles (listes, formulaires). L'objectif est une **expérience prévisible** en cas d'échec.

Q13. Quel est le rôle du widget de chat dans l'expérience utilisateur ?

R. Il propose une **assistance contextuelle** : orientation dans le catalogue, réponses aux questions fréquentes, et lorsque le backend est configuré, des **suggestions de produits** renvoyées avec la réponse pour raccourcir le parcours d'achat.

Q14. Pourquoi Bootstrap, et comment éviter une interface « générique » ?

R. Bootstrap accélère la **responsivité** et les composants de base ; une couche de **thème** (CSS d'override, identité ASMAD) personnalise l'apparence tout en gardant une maintenance simple.

4. Couche application (NestJS, API)

Q15. Comment découpez-vous fonctionnellement le backend en modules ?

R. Chaque domaine métier ou transverse a son module : **User, Auth, Category, Product, Review, Cart, Order, Upload, Mail, SellerFollow, Chat, Dashboard**, etc., déclarés dans `AppModule`. Cela respecte le principe de **cohésion forte** et de **couplage faible** entre domaines.

Q16. Comment externalisez-vous la configuration (secrets, connexion base, clés API) ?

R. **@nestjs/config** charge un fichier `.env` (non versionné). Un `.env.example` sert de modèle documenté pour recréer un environnement local ou de préproduction sans exposer de secrets.

Q17. Sur quel mécanisme repose la validation des entrées HTTP ?

R. Des **DTO** annotés avec **class-validator** et **class-transformer**, combinés aux pipes NestJS : les données invalides sont rejetées **avant** l'exécution du service, ce qui réduit les risques d'états incohérents.

Q18. Quel est le rôle du `DatabaseInitService` dans votre démarche ?

R. Il s'agit typiquement d'un **amorçage** au démarrage (jeux de données minimaux, comptes de démonstration, référentiels) pour rendre l'environnement de développement ou de soutenance **immédiatement exploitable**. Précisez à l'oral le contenu exact généré dans votre branche.

5. Modèle de données et règles métier

Q19. Pourquoi un SGBDR (MySQL) et un ORM (TypeORM) pour ce type d'application ?

R. Les entités (utilisateurs, commandes, lignes de commande, panier) se prêtent naturellement au modèle **relationnel** (intégrité référentielle, transactions). TypeORM aligne le **modèle objet** TypeScript sur le schéma SQL et limite les requêtes SQL ad hoc non maîtrisées.

Q20. Que signifie `synchronize: true` dans TypeORM, et quelle est votre position pour la production ?

R. TypeORM **modifie le schéma** pour refléter les entités : pratique en développement, **dangereux en production** (migrations implicites, risque de perte). En entreprise, on bascule vers des **migrations versionnées** et `synchronize: false`.

Q21. Comment modélisez-vous produit, catégorie et médias associés ?

R. **Product** est relié à **Category** ; les images sont portées par une entité **Image** (lien au produit, ordre d'affichage). Un **statut produit** (`ProductStatus`) permet de gérer le cycle de vie (brouillon, publié, etc.) selon votre énumération.

Q22. Comment le panier est-il modélisé et pourquoi l'attacher à un utilisateur authentifié ?

R. Des **CartItem** référencent l'utilisateur et le produit avec une quantité. L'association au compte garantit la **persistence** entre sessions et simplifie la transformation en commande.

Q23. Pourquoi distinguer `Order` et `OrderItem` ?

R. Pour **figer l'histoire commerciale** : chaque ligne conserve produit, quantité et prix au moment de la commande, indépendamment des futures modifications du catalogue ou des tarifs.

Q24. Comment garantir la cohérence lors de la création d'une commande à partir du panier ?

R. Idéalement : transaction base de données (tout réussit ou tout est annulé), vérification des **stocks** et des **prix** au moment de la validation, puis vidage ou mise à jour du panier. Détaillez ce que votre `OrderService` fait effectivement.

6. Sécurité : authentification, autorisation, menaces

Q25. Quelle politique appliquez-vous au stockage des mots de passe ?

R. Hachage avec **bcrypt** (coût adapté) : aucun mot de passe en clair en base ; en cas de fuite, les empreintes restent difficiles à inverser.

Q26. Expliquez le flux JWT de bout en bout.

R. Après vérification email/mot de passe, le serveur signe un **JWT** (`@nestjs/jwt`) contenant les claims nécessaires (identité, rôle). Le client le stocke et l'envoie dans l'en-tête `Authorization`. Chaque route protégée valide la signature et l'expiration via **Passport**.

Q27. Distinction entre authentification et autorisation : où le code l'exprime-t-il ?

R. **Authentification** : « qui êtes-vous ? » (JWT valide). **Autorisation** : « que pouvez-vous faire ? » (rôle admin, vendeur, client) via guards ou contrôles dans le service. Une erreur fréquente est de vérifier uniquement côté interface.

Q28. Comment sécurisez-vous la réinitialisation de mot de passe ?

R. Génération d'un **code à durée de vie limitée** stocké en table dédiée (`ResetCode`), envoi par e-mail (**Nodemailer**), vérification puis mise à jour du hash. Il faut aussi limiter le **brute force** sur le code (piste d'amélioration : rate limiting).

Q29. Qu'est-ce que la logique « première connexion » liée au CIN ?

R. Règle métier : le mot de passe initial peut correspondre au **CIN** ; le système détecte ce cas et peut imposer un **changement obligatoire** pour réduire le risque lié à un secret faible et connu administrativement.

Q30. Quelles contre-mesures proposez-vous face à XSS, injection SQL et abus d'API ?

R. **XSS :** templates Vue qui échappent par défaut, validation des entrées. **Injection SQL :** ORM et requêtes paramétrées. **Abus :** limitation de débit, CORS strict, HTTPS en production, rotation des secrets, journalisation des anomalies. Mentionnez ce qui est **déjà en place** et ce qui est **prévu**.

7. Parcours d'achat, panier, commande

Q31. Décrivez le parcours critique « découverte → commande » du point de vue données.

R. Navigation catalogue → consultation fiche produit → ajout au **panier** (lignes persistées) → passage par le **checkout** → création d'une **commande** et de ses lignes → consultation dans « Mes commandes ».

Q32. Où en est l'intégration d'un prestataire de paiement (PSP) ?

R. Réponse transparente : si la démo s'arrête à un **statut de commande** ou une simulation, le jury attend une vision claire des **étapes d'intégration** (redirection, webhooks, idempotence, conformité PCI-DSS en encadrant ce qui reste côté PSP).

Q33. Comment traitez-vous la question des stocks et des commandes concurrentes ?

R. Si le champ existe : décrément atomique ou verrouillage optimiste/pessimiste en transaction. Sinon : reconnaître la limite actuelle et proposer une **file d'attente** ou une réservation temporaire comme évolution.

8. Fichiers, images (sans Cloudinary), e-mail, suivi vendeurs

Q34. Utilisez-vous un service tiers type Cloudinary pour les images ?

R. **Non.** Le projet ne s'appuie pas sur Cloudinary pour l'hébergement ou la transformation d'images. (Si des paquets npm liés à Cloudinary figurent encore dans `package.json`, vous pouvez préciser qu'ils ne sont **pas utilisés en production** dans votre chaîne actuelle — évitez toute ambiguïté avec le jury.)

Q35. Alors, comment les images sont-elles transmises et stockées dans votre implémentation ?

R. L'endpoint `POST /upload/file` reçoit le fichier via **Multer** (`FileInterceptor`), vérifie le type MIME (images), convertit le buffer en **chaîne base64** et renvoie une **data URL** (`…`). Le service d'**images** peut ensuite enregistrer le contenu binaire en base (entité **Image**, champ buffer) avec contrôle de taille (plafond côté serveur). Ainsi, pas de CDN externe : les médias transitent par votre API et sont persistés dans **MySQL**.

Q36. Quels sont les compromis du stockage d'images en base (BLOB) par rapport à un stockage objet (S3, disque) ?

R. **Avantages :** simplicité de sauvegarde/restauration avec la base, cohérence transactionnelle. **Inconvénients :** base plus lourde, mise en cache et CDN moins triviales, coût des grosses pièces jointes. En évolution, on pourrait externaliser les fichiers et ne garder en base que l'URL ou la clé.

Q37. Quel est le rôle du module Mail et quels flux l'alimentent ?

R. **Nodemailer** envoie des e-mails transactionnels via SMTP (paramètres `.env`), notamment pour la **réinitialisation de mot de passe** et d'éventuelles notifications métier.

Q38. Comment modélisez-vous la relation « client qui suit un vendeur » ?

R. Le module **seller-follow** matérialise une association utilisateur → vendeur, exposée en API pour alimenter les écrans de **marchés suivis** ou d'abonnements, et favoriser la rétention.

9. Assistance conversationnelle et IA (OpenAI)

Q39. Quelle valeur métier apporte l'intégration d'un modèle de langage à votre marketplace ?

R. Elle complète la recherche classique par une **interaction naturelle** : reformulation des besoins, explications, et mise en avant de produits pertinents dans le contexte du catalogue réel.

Q40. Comment le backend relie-t-il la réponse du modèle au catalogue ?

R. Le `ChatService` s'appuie sur les **produits** en base ; un service d'**embeddings** peut scorer la similarité sémantique ; la réponse inclut du texte et une liste structurée de **produits recommandés** pour l'affichage dans le widget.

Q41. Que se passe-t-il si la clé OpenAI est absente ou si le service externe est indisponible ?

R. Le service doit échouer proprement (**503** ou message explicite) : le chat est une **fonction optionnelle** ; le cœur e-commerce reste utilisable sans IA.

Q42. Quelles précautions RGPD et de gouvernance associez-vous à l'usage de l'IA ?

R. Minimisation des données personnelles dans les prompts, information de l'utilisateur, durée de conservation des journaux, et distinction entre **données traitées par un sous-traitant** (OpenAI) et données strictement locales.

10. Pilotage, tableaux de bord, indicateurs

Q43. Quels indicateurs exposez-vous aux administrateurs ou vendeurs ?

R. Le **DashboardModule** agrège des métriques (volumes, tendances selon implémentation) ; le front les visualise avec **Chart.js** / **vue-chartjs** pour une lecture rapide de l'activité.

Q44. À quoi sert le module `forecast` dans votre architecture ?

R. Il peut fournir des **indicateurs prédictifs simples** ou des agrégations temporelles pour enrichir le tableau de bord ou le chat. Précisez à l'oral la formule ou la logique réellement codée (moyenne mobile, extrapolation, etc.).

11. Qualité logicielle, performance, industrialisation

Q45. Quelle est votre stratégie de tests : unitaires, intégration, e2e ?

R. NestJS fournit **Jest** ; décrivez ce que vous avez couvert (services critiques, contrôleurs mockés). Si la couverture est limitée, présentez un **plan priorisé** : auth, création de commande, panier.

Q46. Outre les tests, quels garde-fous utilisez-vous pour la qualité ?

R. **ESLint**, typage TypeScript, conventions de dossiers, revues entre pairs, et commits réguliers pour tracer l'historique des décisions.

Q47. Si le trafic multiplie par dix, quels seraient vos premiers leviers ?

R. Mise en cache lecture (Redis), **pagination** systématique des listes, index SQL sur clés de jointure et filtres, mise en **CDN** des assets statiques du front, montée en charge horizontale du backend derrière un load balancer, lecture répliquée MySQL.

12. Méthodologie agile (Scrum) et collaboration

Q48. Comment avez-vous appliqué Scrum (time-boxing, artefacts, cérémonies) ?

R. Sprints de durée fixe, **backlog produit** priorisé, objectifs de sprint, revue de fin de sprint et rétrospective. Vos livrables dans `docs/` (rapports, epics, planning) peuvent illustrer cette démarche.

Q49. Comment avez-vous découpé et priorisé les fonctionnalités ?

R. Par **epics** (identité, catalogue, transaction, espaces métiers, IA) puis **user stories** avec critères d'acceptation testables ; priorisation par **valeur utilisateur** et **risque technique**.

Q50. Quelle politique de branches et d'intégration continue suivez-vous ?

R. **Git** sur `main` (ou branches de fonctionnalités si équipe) ; commits atomiques ; possibilité de CI (build + lint) sur push — à confirmer selon votre dépôt distant.

13. Déploiement, exploitation, continuité de service

Q51. Décrivez un déploiement de référence pour le front et le back.

R. Front : `vue-cli-service build`, fichiers statiques servis par **Nginx** ou hébergeur statique. Back : `nest build`, process Node supervisé (PM2, systemd), base **MySQL** managée, secrets injectés par l'environnement d'hébergement.

Q52. Comment sécuriseriez-vous les sauvegardes et la reprise après sinistre ?

R. Dumps MySQL planifiés, chiffrement au repos, tests de restauration périodiques, et définition d'un **RPO/RTO** cible selon l'enjeu métier.

14. Bilan critique et feuille de route

Q53. Quelles limites assumez-vous franchement sur la version soutenue ?

R. Exemples : pas de PSP complet, schéma géré par `synchronize` en dev, images en BLOB (scalabilité à revoir), dépendance à un fournisseur IA externe, monitoring applicatif à renforcer.

Q54. Quelle feuille de route sur 6 à 12 mois ?

R. Paiement réel, stockage objet pour médias, recherche full-text ou moteur dédié, application mobile, messagerie vendeur-client, promotions, internationalisation complète, observabilité (logs, métriques, traces).

Q55. En quoi votre architecture facilite-t-elle la maintenance à moyen terme ?

R. Front et back découplés, modules NestJS par domaine, entités et DTO explicites, configuration externalisée, conventions homogènes — ce qui réduit le coût des évolutions pour une petite équipe.

Q56. Si vous recommenciez le projet, que feriez-vous différemment dès le premier sprint ?

R. Migrations TypeORM versionnées dès le début, jeux de tests e2e sur le parcours commande, maquettage utilisateur plus tôt, et décision explicite sur la stratégie médias (objet vs BLOB) pour éviter une dette technique.

Q57. Comment réagiriez-vous si le jury vous demande de justifier un choix technique contesté (ex. images en base) ?

R. Exposer le **contexte** (PFE, délai, simplicité de sauvegarde), les **compromis**, puis la **voie d'évolution** documentée : c'est une réponse de niveau ingénieur, pas une excuse.

Q58. Quelle est la proposition de valeur de votre démo par rapport à une solution SaaS clé en main ?

R. Maîtrise du code, adaptation aux règles locales (CIN, rôles), traçabilité des choix d'architecture, et capacité à étendre précisément les modules (dashboard, IA, suivi vendeurs) sans dépendre d'un no-code fermé.

Personnalisation : croisez chaque réponse avec votre démo (écrans, endpoints réellement testés, champs d'entités). Les questions Q18, Q24, Q33, Q44 et Q45 appellent une réponse ancrée dans *votre* code.

PDF : Fichier → Imprimer → Enregistrer au format PDF depuis le navigateur, ou Chrome `--headless --print-to-pdf` sur ce HTML.